

Assignment 2: GCD of two integers

*Instructor: Subhamoy Moitra**Written by: Sreejit Maity*

1 Introduction

GCD means Greatest Common Divisor. Given two integers, their GCD is the largest possible integer value that divides both the numbers.

The interesting thing about GCD is around its divisibility. The difference of two integers must be their GCD or a multiple of their GCD. Similarly, Division algorithm gives the remainder part as GCD of the two numbers or a multiple of their GCD. We use these results to write code for GCD of two numbers rigorously or recursively.

2 Finding GCD of two integers in C

```
unsigned int gcd(unsigned int a, unsigned int b) {  
    while (b != 0) {  
        unsigned int temp = b;  
        b = a % b;  
        a = temp;  
    }  
    return a;  
}
```

The definition of GCD is the nonzero number, when there are given one nonzero number and another number being zero. Also, $\text{gcd}(0, 0) = 0$.

3 Correctness

Lemma 3.1 (Correctness of the Euclidean GCD algorithm). *For all unsigned integers a, b not both zero, the function call $\text{gcd}(a, b)$ returns $\text{gcd}(a, b)$.*

Proof. We first establish the key mathematical fact the algorithm relies on, and then use it to prove a loop invariant.

Claim. For any integers a, b with $b \neq 0$,

$$\text{gcd}(a, b) = \text{gcd}(b, a \bmod b).$$

Proof of Claim. Let $r = a \bmod b$, so $a = qb + r$ for some integer q . Any common divisor d of a and b must also divide $a - qb = r$, so d is a common divisor of b and r . Conversely, any common divisor d of b and r must divide $qb + r = a$, so d is a common divisor of a and b . Hence a and b have exactly the same set of common divisors as b and r , so in particular their greatest common divisors agree: $\text{gcd}(a, b) = \text{gcd}(b, r)$.

Loop invariant. We claim that at the start of every iteration of the **while** loop, $\text{gcd}(a, b) = \text{gcd}(a_0, b_0)$, where a_0, b_0 denote the original input values.

Base case. Before the first iteration, $a = a_0$ and $b = b_0$, so the invariant holds trivially.

Inductive step. Suppose the invariant holds at the start of some iteration, i.e. $\text{gcd}(a, b) = \text{gcd}(a_0, b_0)$, and suppose $b \neq 0$ (so the loop body executes). The loop body computes **temp** = **b**, then **b** = **a % b**, then **a** = **temp**, which corresponds to the simultaneous update

$$(a, b) \longleftarrow (b, a \bmod b).$$

By the Claim above, $\text{gcd}(b, a \bmod b) = \text{gcd}(a, b)$, which by the inductive hypothesis equals $\text{gcd}(a_0, b_0)$. Hence the invariant is preserved after the update, and holds again at the start of the next iteration.

Termination. Each iteration replaces b with $a \bmod b$, which satisfies $0 \leq a \bmod b < b$. Thus the value of b is a non-negative integer that strictly decreases at every iteration (as

long as $b \neq 0$), so the loop cannot run forever and must terminate after finitely many steps, exactly when b reaches 0.

Conclusion. When the loop terminates, $b = 0$, and by the invariant, $\gcd(a, b) = \gcd(a, 0) = a = \gcd(a_0, b_0)$, since $\gcd(a, 0) = a$ for any $a > 0$. At this point the function returns **a**, which is therefore exactly $\gcd(a_0, b_0)$, as required. $\square$

4 Time required to calculate GCD

Theorem 4.1 (Running time of the Euclidean algorithm). *On inputs a, b with $a \geq b > 0$, the `while` loop in `gcd(a, b)` executes $O(\log b)$ iterations, and hence the algorithm runs in $O(\log b)$ time, where each iteration is assumed to take constant time aside from the cost of the modulo operation itself.*

Proof. We show that two consecutive iterations reduce the value of b by at least a constant factor, which bounds the total number of iterations logarithmically.

Key inequality. Suppose at some iteration the pair is (a, b) with $a \geq b > 0$, and let $r = a \bmod b$, so the next pair is (b, r) . We claim

$$r < \frac{a}{2}.$$

To see this, consider two cases. If $b \leq a/2$, then since $0 \leq r < b$, we immediately get $r < b \leq a/2$. If instead $b > a/2$, then $a = 1 \cdot b + r$ (the quotient is exactly 1), so $r = a - b < a - a/2 = a/2$. In either case, $r < a/2$.

Consequence. Applying this inequality every two iterations, the larger of the two values in the pair at least halves:

$$(a, b) \rightarrow (b, r) \rightarrow (r, r') \rightarrow \dots$$

so after every two iterations, the current values are less than half of what they were two iterations earlier. Since b starts at most a , after $2k$ iterations the values are $O(a/2^k)$. The

loop terminates once $b = 0$, which by the above must happen once $k = O(\log b)$, i.e. after $O(\log b)$ iterations overall. $\square$

Corollary 4.2. *The Euclidean algorithm runs in time $O(\log(\min(a, b)))$, i.e. logarithmic in the smaller of the two inputs.*

Remark 4.3 (Worst case: Fibonacci numbers). *The bound above is tight. If $a = F_{n+1}$ and $b = F_n$ are consecutive Fibonacci numbers, then every step of the algorithm computes a quotient of exactly 1, since $F_{n+1} = 1 \cdot F_n + F_{n-1}$. Thus the algorithm takes exactly n iterations, and since F_n grows exponentially as $F_n = \Theta(\varphi^n)$ where $\varphi = \frac{1+\sqrt{5}}{2}$ is the golden ratio, this shows the number of iterations is $\Theta(\log_\varphi b)$ in the worst case, matching the $O(\log b)$ upper bound up to constants. This result is known as Lamé's theorem.*

Remark 4.4 (Bit complexity). *If a and b are n -bit unsigned integers, then $\log b = O(n)$, so the number of iterations is $O(n)$. However, each modulo operation $a \% b$ on n -bit numbers itself takes $O(n^2)$ time using schoolbook division (or $O(n \log n)$ with fast multiplication-based division), so the total bit complexity of the algorithm is $O(n^3)$ in the worst case using naive arithmetic. Faster variants, such as the binary GCD algorithm, avoid full division and achieve $O(n^2)$ bit complexity, and asymptotically fast algorithms bring this down to $O(n \log^2 n)$.*

5 Reference

I have done the coding myself with the help of the idea of computing time from Geeks-ForGeeks. The time requirement is taken from Claude and modified by me. The correctness part is mostly web search oriented from various sites.